



ENGINEERING HANDBOOK · 2026

mise 完整使用指南

开发环境、工具链与工程任务的统一管理

从安装、工具版本、环境变量和任务，到锁文件、CI、安全边界与团队落地。

Winston

版本 V1.0 · 2026-06-02

基于 mise v2026.5.18 官方资料

| 目录

01	执行摘要	03
02	1. mise 解决的不是一个问题	04
03	2. 15 分钟上手	05
04	3. mise use、mise install、mise exec 和 mise run	08
05	4. 三种接入方式	09
06	5. mise.toml: 项目级入口	10
07	6. 工具、Registry 与后端	13
08	7. 环境变量: 把依赖写成契约	15
09	8. 任务系统: 给项目一个标准入口	17
10	9. 锁文件: 可读范围与可复现安装同时存在	19
11	10. CI: 让本地和流水线读同一份配置	21
12	11. 安全: 默认配置要克制	23
13	12. 最佳食用方式	25
14	13. 从 asdf 与 direnv 迁移	27
15	14. 排障顺序	28
16	15. 常见误区	30
17	16. 一份可以直接采用的项目模板	31
18	17. Further Reading	33
19	附录: mise Cheatsheet	34
20	附录: 资料来源与边界	44

执行摘要

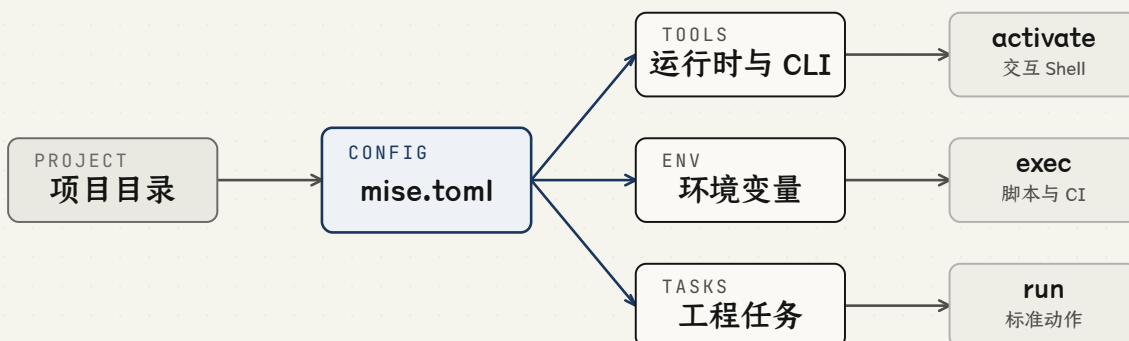
一个项目常常同时依赖 Node.js、Python、Go、Terraform、`ripgrep`、`shellcheck`、环境变量和若干脚本。每项依赖单独管理时，新成员需要先读一页安装说明，CI 还要再写一套初始化逻辑。过几个月后，本地、文档和 CI 往往已经不是同一套版本。

`mise` 把这些约定收回项目目录。官方 README 对它的定位很直接：**dev tools、env vars、tasks in one CLI**。你可以在一个 `mise.toml` 中声明工具、环境变量和任务，让新 Shell、新 checkout 和 CI job 从同一份配置开始工作。

这份教程给出一条偏保守的落地路线：

1. 个人电脑使用 `mise activate` 接入交互 Shell。
2. 用 `mise use` 安装工具并把版本写入 `mise.toml`。
3. 用 `mise exec` 和 `mise run` 处理脚本、任务与 CI。
4. 团队项目启用并提交 `mise.lock`，保留版本范围的可读性，同时锁定真实安装结果。
5. `secret` 不进 `mise.toml`，实验性 hooks 不放进默认配置。

FIGURE 1 MISE PROJECT MODEL



一份项目配置同时约束工具、环境变量和任务。交互 Shell、脚本和 CI 只是不同入口。

Figure 1. 团队只需维护一份项目契约，差异留在入口，不留在安装说明里。

| 1. mise 解决的不是一个问题

mise 发音接近 “meez”，名字来自法语 mise en place，也就是在开始做饭前把工具和材料放到位。这个比喻与工程实践很贴切：你进入项目目录时，运行时、CLI、环境变量和常用任务已经准备好。

1.1 工具版本

不同项目可以使用不同版本的 Node.js、Python、Ruby、Go、Java、Terraform 和其他 CLI。进入目录后，mise 根据当前配置切换 PATH。一个项目可以用 Node.js 24，另一个项目用 Node.js 26，不需要手工切换。

1.2 环境变量

项目级环境变量可以写入 [env]，也可以从 .env、JSON 或 YAML 文件载入。mise 支持 required variables、PATH 扩展和 redaction，适合把“项目运行需要哪些变量”写成显式契约。

1.3 任务

mise run 可以执行构建、测试、lint、部署前检查和本地开发任务。任务能声明依赖关系、输入输出文件和确认提示。多个无依赖任务默认最多并发执行 4 个 job。

对新项目，先把 mise 当成一个项目入口：mise install 准备工具，mise run 执行标准动作。熟悉之后，再逐步使用锁文件、环境分层和安全设置。

2. 15 分钟上手

2.1 安装

官方安装页给出的推荐方式因平台而异：

平台	推荐方式	命令
macOS	Homebrew	<code>brew install mise</code>
Debian / Ubuntu	apt	先添加官方源,再执行 <code>sudo apt install mise</code>
Fedora / RHEL	dnf	<code>sudo dnf copr enable jdxcode/mise && sudo dnf install mise</code>
Arch Linux	pacman	<code>sudo pacman -S mise</code>
Alpine Linux	apk	<code>apk add mise</code>
Windows	Scoop	<code>scoop install mise</code>
CI / Docker	mise.run	<code>curl https://mise.run sh</code>

Linux 和 macOS 想快速试用,可以直接运行：

```
curl https://mise.run | sh
~/.local/bin/mise --version
```

`mise.run` 默认把可执行文件放到 `~/.local/bin/mise`。如果你通过包管理器安装,后续升级优先交给包管理器；如果你通过 `mise.run` 安装,可以使用 `mise self-update`。

2.2 激活交互 Shell

`mise exec` 不依赖 Shell 激活,安装完成后已经可以使用。日常开发更适合启用 `mise activate`,让 `mise` 在目录变化和 Shell prompt 刷新时更新 PATH 与环境变量。

```
# bash
echo 'eval "$($(~/.local/bin/mise activate bash)'" >> ~/.bashrc

# zsh
echo 'eval "$($(~/.local/bin/mise activate zsh)'" >> ~/.zshrc

# fish
echo '~/.local/bin/mise activate fish | source' >> ~/.config/fish/config.fish
```

通过 Homebrew 或其他包管理器安装时,通常可以省略绝对路径：

```
echo 'eval "$(mise activate zsh)"' >> ~/.zshrc
```

重启 Shell 后检查：

```
mise doctor
```

2.3 在项目里安装工具

新建一个目录：

```
mkdir demo-mise
cd demo-mise
mise use node@26
node -v
```

mise 会安装当前 Node.js 26 系列的最新匹配版本，并创建 `mise.toml`：

```
[tools]
node = "26"
```

再加一个 Python：

```
mise use python@3.14
python --version
```

此时 `mise.toml` 类似：

```
[tools]
node = "26"
python = "3.14"
```

2.4 运行一次性命令

不想写配置，只想临时跑一个版本时，用 `mise exec`：

```
mise exec node@26 -- node -v
mise exec python@3.14 -- python -c 'print("hello")'
```

`mise exec` 会在需要时下载工具，然后只在当前命令中注入 mise 环境。

2.5 添加第一个任务

在 `mise.toml` 中加入：

```
[tasks.hello]
description = "Print runtime versions"
run = """
node -v
python --version
"""
```

运行：

```
mise run hello
```

任务会自动带上当前项目的工具和环境变量。到这里，你已经完成了最小闭环。

3. mise use、mise install、mise exec 和 mise run

这 4 个命令承担不同职责。理解它们的边界，可以避开大部分新手问题。

命令	做什么	是否写配置	典型场景
<code>mise use node@26</code>	安装并激活工具	是	为项目新增或调整工具
<code>mise install</code>	安装配置中声明的工具	否	clone 项目后的初始化
<code>mise exec -- <cmd></code>	在 mise 环境中执行命令	否	脚本、CI、一次性命令
<code>mise run <task></code>	运行项目任务	否	构建、测试、lint、部署前检查

官方 Walkthrough 特别提醒: `mise install node@26` 只安装工具，不会让它进入项目 PATH。日常开发新增工具时，优先使用：

```
mise use node@26
```

团队成员拉取已有项目后，运行：

```
mise install
```

脚本或 CI 中不要假设 prompt 会刷新 PATH。使用：

```
mise install
mise exec -- npm test
```

或把标准动作定义为任务：

```
mise install
mise run ci
```


4. 三种接入方式

mise 提供 PATH activation、shims 和显式执行三种接入方式。它们适合不同环境。

场景	推荐方式	原因
个人交互 Shell	<code>mise activate</code>	目录变化时自动更新 PATH 和环境变量
CI、IDE、非交互脚本	<code>mise exec</code> 、 <code>mise run</code> 或 <code>shims</code>	不依赖 Shell prompt
只在单个项目里使用	<code>mise exec</code> 和 <code>mise run</code>	无需修改 Shell rc 文件

4.1 PATH activation

`mise activate` 会在 prompt 显示或目录变化时更新环境。执行 `which node` 时，你会看到真实安装路径，而不是 shim：

```
which node
# ~/.local/share/mise/installs/node/26/bin/node
```

官方文档建议交互场景优先使用这种方式。

4.2 Shims

shims 是位于 `~/.local/share/mise/shims` 的轻量入口。它们适合没有交互 prompt 的环境：

```
export PATH="$HOME/.local/share/mise/shims:$PATH"
```

shims 有明确限制：

- `[env]` 中的变量只会在调用 mise 管理的工具时可用。
- 多数 `cd`、`enter`、`leave` 和 `watch_files hooks` 不会触发。
- `which node` 会指向 shim，需要使用 `mise which node` 查看真实路径。

4.3 显式执行

`mise exec` 和 `mise run` 会在执行前加载工具与环境变量。对于 CI 和脚本，这种方式最容易读懂：

```
mise exec -- npm test
mise run lint
```

| 5. mise.toml: 项目级入口

一个典型项目配置可以从下面这份文件开始：

```
min_version = "2026.5.18"

[tools]
node = "26"
python = "3.14"
uv = "latest"
ripgrep = "latest"

[env]
NODE_ENV = "development"
_.path = "./node_modules/.bin"

[tasks.install]
description = "Install project dependencies"
run = [
    "npm install",
    "uv sync",
]

[tasks.test]
description = "Run tests"
run = [
    "npm test",
    "uv run pytest",
]

[tasks.lint]
description = "Run linters"
run = [
    "npm run lint",
    "uv run ruff check .",
]

[tasks.ci]
description = "Run CI checks"
depends = ["lint", "test"]
```

`min_version` 很适合团队项目。配置使用了新字段时，旧版 mise 会明确报错或提醒升级，避免默默忽略行为差异。

5.1 配置会向父目录递归

mise 从当前目录向上查找配置，并把它们合并起来。更接近当前目录的配置覆盖上层配置。

```
~/.config/mise/config.toml
~/work/mise.toml
~/work/project/mise.toml
~/work/project/mise.local.toml
~/work/project/backend/mise.toml
```

进入 `backend/` 时, `mise` 会同时读取这些层级。你可以用下面的命令查看实际生效文件:

```
mise config
```

或:

```
mise config ls
```

5.2 共享配置和本地覆盖分开

提交到仓库:

```
mise.toml
mise.lock
```

加入 `.gitignore` :

```
mise.local.toml
mise.local.lock
mise.*.local.toml
mise.*.local.lock
```

本机 `secret`、个人工具和临时设置放在 `local` 文件里, 不要污染共享配置。

5.3 环境专用配置

需要区分开发、测试和生产工具时, 可以使用 `MISE_ENV` :

```
MISE_ENV=test mise install
mise -E test run test
```

`mise` 会按环境加载:

```
mise.toml
mise.test.toml
mise.local.toml
mise.test.local.toml
```

多个环境可以组合：

```
MISE_ENV=ci,test mise run test
```

后出现的环境优先级更高。

5.4 .tool-versions 和 idiomatic version files

mise 可以读取 asdf 使用的 .tool-versions 。已有 asdf 项目可以先直接使用，再逐步迁移到 mise.toml 。

mise 也支持语言生态常见的版本文件，例如：

工具	常见文件
Node.js	.nvmrc 、 .node-version 、 package.json
Python	.python-version 、 .python-versions
Ruby	.ruby-version
Go	go.mod
Java	.java-version

按工具开启读取：

```
mise settings add idiomatic_version_file_enable_tools node
mise settings add idiomatic_version_file_enable_tools python
```

这适合协作方不统一使用 mise 的仓库：项目继续保留生态通用文件，mise 用户仍能获得自动切换。

| 6. 工具、Registry 与后端

6.1 Registry 提供 shorthand

输入：

```
mise use aws-cli
```

Registry 会把 shorthand 映射到实际后端，例如：

```
mise use aqua:aws/aws-cli
```

查看可用工具：

```
mise registry  
mise search ripgrep  
mise tool ripgrep
```

6.2 后端是什么

后端负责发现版本、下载、安装和配置工具。官方后端列表包括：

```
asdf, aqua, cargo, conda, dotnet, forgejo, gem,  
github, gitlab, go, http, npm, pipx, s3, spm,  
ubi, vfox, custom backends
```

部分后端带实验性标记。Registry 文档给出的新 entry 优先级很有参考价值：

1. 优先使用 `aqua`。
2. 工具不在 `aqua registry` 时，考虑 `github` 或 `gitlab`。
3. 只有工具无法合理通过前两类后端支持时，再考虑 `conda`、`pipx`、`npm`、`gem`、`go`、`cargo` 或 `dotnet`。
4. 新 `vfox` 与 `asdf registry entry` 不再接受，原因是供应链风险。

用户仍可显式使用任何后端：

```
mise use github:BurntSushi/ripgrep  
mise use npm:prettier  
mise use pipx:black  
mise use cargo:starship
```

6.3 版本范围与升级

项目配置通常写可读范围：

```
[tools]
node = "26"
python = "3.14"
```

安装当前范围内最新版本：

```
mise install
```

查看过期工具：

```
mise outdated
```

在现有范围内升级：

```
mise upgrade
mise upgrade node
```

提升主版本范围：

```
mise upgrade --bump node
```

例如，`mise.toml` 写着 `node = "24"` 时，如果解析结果升级到 Node.js 26，`mise upgrade --bump node` 会把配置改到 `node = "26"`。

7. 环境变量：把依赖写成契约

7.1 基础变量

```
[env]
NODE_ENV = "development"
RUST_BACKTRACE = "1"
```

也可以用 CLI：

```
mise set NODE_ENV=development
mise set
mise unset NODE_ENV
```

7.2 PATH 扩展

项目脚本经常需要把本地目录加入 PATH：

```
[env]
_.path = "./node_modules/.bin"
```

相对路径以项目的 `config_root` 为准。即使你在子目录运行任务，路径仍然稳定。

7.3 读取 `.env`、JSON 和 YAML

```
[env]
_.file = ".env"
```

多个文件：

```
[env]
_.file = [
  ".env",
  ".env.local",
  { path = ".secrets.yaml", redact = true },
]
```

`env._.file` 支持 `dotenv`、JSON 和 YAML。它适合载入文件，不代表这些文件应该提交到仓库。

7.4 Required variables

共享配置可以声明必须由用户提供的变量：

```
[env]
DATABASE_URL = { required = "Set DATABASE_URL in mise.local.toml" }
API_KEY = { required = "Get API_KEY from the team secret manager" }
```

当变量缺失时, 常规命令会报错。Shell activation 会提示警告, 但不会直接破坏 Shell 启动。

7.5 Redaction

```
redactions = ["SECRET_*", "*_TOKEN", "PASSWORD"]

[env]
API_TOKEN = { value = "token-value", redact = true }
```

redaction 通过拦截任务输出实现。raw = true 的任务直接连接标准输入输出, 无法应用 redaction。GitHub Actions 中如果直接使用 mise, 可以把 redacted values 加入 mask:

```
for value in $(mise env --redacted --values); do
  echo "::add-mask::$value"
done
```

jdx/mise-action 会自动处理标记为 redacted 的值。

redaction 只降低日志泄漏概率。secret 仍应来自本机 local 文件、系统环境变量或组织 secret manager。不要把真实密钥提交进 mise.toml。

| 8. 任务系统: 给项目一个标准入口

8.1 TOML 任务

简单任务可以写成一行:

```
[tasks]
build = "npm run build"
test = "npm test"
lint = "npm run lint"
```

复杂任务使用独立表:

```
[tasks.test]
description = "Run unit and integration tests"
run = [
  "npm test",
  "uv run pytest",
]
```

运行:

```
mise run test
```

脚本和文档中始终写完整的 `mise run <task>`。虽然 `mise test` 可能可用, 但未来 `mise` 新增同名 CLI 命令时会遮蔽任务。

8.2 文件任务

逻辑复杂时, 把任务写成可执行脚本:

```
mkdir -p mise-tasks
touch mise-tasks/build
chmod +x mise-tasks/build
```

```
#!/usr/bin/env bash
#MISE description="Build the project"
set -euv pipefail
npm run build
```

文件任务保留 Shell 高亮、lint 和调试体验, 适合多行流程。

8.3 依赖图与并发

```
[tasks.lint]
run = "npm run lint"

[tasks.test]
run = "npm test"

[tasks.build]
run = "npm run build"

[tasks.ci]
depends = ["lint", "test", "build"]
```

`lint`、`test` 和 `build` 之间没有依赖, `mise` 会在 `job` 限制内并发执行。默认最大并发数是 4。

```
mise run --jobs 2 ci
MISE_JOBS=2 mise run ci
```

8.4 只在输入变化时重跑

```
[tasks.build]
description = "Build TypeScript"
run = "npm run build"
sources = [
  "src/**/*.ts",
  "!src/**/*.test.ts",
  "tsconfig.json",
]
outputs = ["dist/**/*.js"]
```

当输出比输入新时, `mise` 会跳过任务。 `sources` 也会被 `mise watch` 使用:

```
mise use --global watchexec@latest
mise watch build
```

8.5 危险任务加确认

```
[tasks.deploy]
description = "Deploy production"
confirm = { message = "Deploy production?", default = "no" }
run = "./scripts/deploy.sh"
```

注意: `confirm` 只保护当前任务自己的 `run`。 `depends` 会在确认前执行。如果依赖也有副作用, 需要给依赖任务单独加确认, 或改成结构化 `run`。

9. 锁文件：可读范围与可复现安装同时存在

9.1 为什么要用 `mise.lock`

`mise.toml` 适合表达意图：

```
[tools]
node = "26"
python = "3.14"
```

`mise.lock` 适合表达一次已经解析过的真实结果。官方 `lockfile` 文档列出 4 个作用：

- 固定团队和 CI 实际安装的版本。
- 在后端支持时保存 checksum。
- 保留 URL，减少重复查询 GitHub 等服务。
- 在支持的后端上记录 provenance 信息。

9.2 启用和生成

```
mise settings lockfile=true
mise lock
mise install
git add mise.toml mise.lock
git commit -m "chore: lock mise toolchain"
```

也可以在项目配置中启用：

```
[settings]
lockfile = true
```

一旦锁文件存在，`mise install`、`mise use` 和 `mise upgrade` 会维护它。

9.3 团队日常流程

新增或调整工具：

```
mise use node@26
mise install
git add mise.toml mise.lock
```

范围内升级：

```
mise upgrade
git add mise.lock
```

提升主版本:

```
mise upgrade --bump node
git add mise.toml mise.lock
```

9.4 Strict lockfile mode

CI 需要避免临时解析外部 API 时, 可以使用:

```
MISE_LOCKED=1 mise install
```

或:

```
[settings]
locked = true
```

strict mode 会要求当前平台的工具已经在 lockfile 中有可用 URL。多平台项目可以预先生成平台条目:

```
mise lock --platform linux-x64,macos-arm64
```

9.5 锁文件不是万能的

后端能力不同。官方文档把支持程度分成多个层次:

后端	lockfile 能力概览
aqua 、 http 、 github 、 gitlab	version、checksum、size、URL
vfox	version、URL、provenance 的部分支持
ubi	version、checksum、size 的部分支持
部分 core 工具	version、checksum
asdf 、 npm 、 cargo 、 pipx	version 为主

锁文件能提高复现性, 但不会自动抹平后端差异。

10. CI: 让本地和流水线读同一份配置

10.1 通用 CI

```
script: |
  curl https://mise.run | sh
  mise install
  mise exec -- npm test
```

如果仓库提交了 bootstrap script, 也可以避免每次动态下载:

```
mise generate bootstrap -l -w
```

CI 中执行:

```
script: |
  ./bin/mise install
  ./bin/mise exec -- npm test
```

10.2 GitHub Actions

截至 2026-06-02, jdx/mise-action 最新 release 是 v4.0.1。action 仓库 README 使用 @v4:

```
name: test
on:
  pull_request:
  push:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v6
      - uses: jdx/mise-action@v4
      - run: mise run ci
```

当仓库中存在 `mise.lock` 时, action 会自动执行 `mise install --locked`。它也会默认启用缓存, 并使用 `${{ github.token }}` 处理 GitHub API 认证。

对供应链要求更严格的组织, 第三方 action 版本应按组织策略 pin 到 commit SHA, 并由依赖更新工具持续维护。

10.3 缓存边界

GitHub Actions 官方文档把缓存定位为重复使用不常变化的文件。mise-action 会基于平台、版本与配置文件 hash 生成缓存键。mise.lock 稳定之后, CI 不需要为每次安装重复解析远端版本, 失败面也更小。

11. 安全：默认配置要克制

11.1 配置信任

`mise.toml` 可以影响环境，也可能包含执行代码的配置。`mise` 会检查配置文件是否已信任：

```
mise trust
mise trust --show
mise untrust
```

在正常模式下，检测到 CI 时配置通常会被视为可信；`paranoid mode` 会收紧这一行为。

11.2 Paranoid mode

```
mise settings paranoid=1
```

或：

```
MISE_PARANOID=1 mise install
```

`paranoid mode` 会要求更严格的配置 `trust`，强制 HTTPS，并在安装时重新验证 `lockfile` 中已有的 `provenance`。它适合高要求环境，不必作为所有开发者的默认设置。

11.3 Minimum release age

刚发布的软件版本留给供应链攻击和误发布的观察时间：

```
[settings]
minimum_release_age = "7d"
```

显式 `pin` 的版本可以绕过 `fuzzy resolution` 的等待窗口。对紧急安全修复，可以在工具级配置更短窗口。

11.4 Hooks 保持审慎

`hooks` 在官方文档中带实验性标记。它们可以在进入目录、离开目录、安装工具或文件变化时执行脚本：

```
[hooks]
enter = "mise install -q"
```

这很方便，也意味着进入目录可能触发代码执行。团队默认配置应保持克制：

- 优先使用显式 `mise run setup`。
- hook 只做耗时短、可预期、无破坏性的动作。
- 不把部署、数据库迁移或 secret 写入放进 hook。

12. 最佳食用方式

12.1 个人开发者

先装 mise 并启用 `mise activate`。全局只保留少量通用工具：

```
mise use --global node@26 python@3.14 uv@latest ripgrep@latest
```

项目工具尽量写进项目的 `mise.toml`。版本差异属于项目事实，不属于个人偏好。

12.2 团队项目

团队仓库至少提交：

```
mise.toml  
mise.lock
```

把常见动作整理成：

```
mise install  
mise run test  
mise run lint  
mise run ci
```

README 只需要解释入口，不需要再维护一页按操作系统分叉的工具安装说明。

12.3 多语言仓库

父目录放共享工具，子目录放服务差异：

```
repo/  
  mise.toml  
  frontend/  
    mise.toml  
  backend/  
    mise.toml
```

父目录：

```
[tools]  
ripgrep = "latest"  
shellcheck = "latest"
```

frontend/mise.toml：

```
[tools]
node = "26"
```

backend/mise.toml :

```
[tools]
python = "3.14"
uv = "latest"
```

12.4 不统一使用 mise 的协作项目

保留生态通用版本文件，例如 `.nvmrc` 和 `.python-version`，本机开启 idiomatic version files。这样 mise 用户可以自动切换，其他开发者继续使用熟悉工具。

| 13. 从 asdf 与 direnv 迁移

13.1 从 asdf 迁移

mise 可以读取 `.tool-versions`，也能在需要时使用 `asdf plugins`。迁移不必一步到位：

1. 安装 mise。
2. 保留已有 `.tool-versions`。
3. 运行 `mise install`。
4. 新增工具时逐步改用 `mise use`。
5. 需要更完整配置时，迁移到 `mise.toml`。

生成配置：

```
mise config generate
```

asdf 官方文档说明 asdf 通过 shims 在执行时解析 `.tool-versions`。mise 交互模式更偏向直接更新 PATH，同时保留 shims 给非交互环境使用。迁移时不要假设两个工具的数据目录可以直接复用。

13.2 direnv 边界

direnv 与 mise 都会通过 Shell hook 管理环境变量。mise 官方文档的建议很明确：不要让两者同时管理同一套 PATH。简单的无关变量可能可以共存，PATH、Python virtualenv 和工具版本容易产生覆盖顺序问题。

已有 direnv 项目可以先做两件事：

1. 让 mise 管理工具版本。
2. 逐步把可迁移环境变量移入 `[env]`，减少 `.envrc` 中的 PATH 操作。

14. 排障顺序

14.1 先跑 doctor

```
mise doctor
mise --version
mise config
mise ls --current
```

14.2 工具版本不对

```
which -a node
mise which node
mise ls node
```

如果 `node` 的第一个路径不在 `mise installs` 或 `shims` 下，检查 Shell rc 文件和 PATH 顺序。脚本中直接用：

```
mise exec -- node -v
```

14.3 新版本看不到

```
mise cache clear
mise ls-remote node
```

`mise` 会缓存版本列表，刚发布的版本可能暂时不可见。必要时再检查 GitHub token、网络与后端状态。

14.4 Prompt 变慢

```
mise deactivate
MISE_TIMINGS=1 mise hook-env -s bash 2>&1 >/dev/null
```

常见原因包括昂贵的 `_.source` 脚本、工具数量过多和依赖网络的环境指令。

14.5 CI 中工具找不到

不要依赖 `mise activate` 等待 prompt 刷新。使用：

```
mise install  
mise exec -- my-tool --version
```

或：

```
export PATH="$HOME/.local/share/mise/shims:$PATH"
```

15. 常见误区

误区	修正
执行 <code>mise install node@26</code> 后期待项目自动切换	新增项目工具使用 <code>mise use node@26</code>
在 CI 中只写 <code>mise activate</code>	使用 <code>mise exec</code> 、 <code>mise run</code> 或 <code>shims</code>
把 <code>secret</code> 写进共享 <code>mise.toml</code>	使用 <code>local</code> 文件、系统环境变量或 <code>secret manager</code>
看到 <code>redact = true</code> 就认为 <code>secret</code> 已安全	<code>redaction</code> 只降低日志暴露风险
同时让 <code>direnv</code> 和 <code>mise</code> 管理 <code>Python PATH</code>	只保留一个 <code>PATH owner</code>
在脚本中用 <code>mise test</code>	写完整的 <code>mise run test</code>
把 <code>hooks</code> 当成默认初始化方式	先用显式 <code>mise run setup</code>
只提交 <code>mise.toml</code> ，忽略团队复现	团队项目生成并提交 <code>mise.lock</code>

16. 一份可以直接采用的项目模板

```
min_version = "2026.5.18"

[settings]
lockfile = true
minimum_release_age = "7d"

[tools]
node = "26"
python = "3.14"
uv = "latest"
ripgrep = "latest"

[env]
NODE_ENV = "development"
DATABASE_URL = { required = "Set DATABASE_URL in mise.local.toml" }
_.path = "./node_modules/.bin"

[tasks.setup]
description = "Install dependencies"
run = [
    "npm install",
    "uv sync",
]

[tasks.lint]
description = "Run linters"
run = [
    "npm run lint",
    "uv run ruff check .",
]

[tasks.test]
description = "Run tests"
run = [
    "npm test",
    "uv run pytest",
]

[tasks.ci]
description = "Run CI checks"
depends = ["lint", "test"]
```

初始化：

```
mise trust
mise lock
mise install
mise run setup
mise run ci
```

本机 secret:

```
# mise.local.toml
[env]
DATABASE_URL = "postgres://localhost/example"
```

.gitignore :

```
mise.local.toml
mise.local.lock
mise.*.local.toml
mise.*.local.lock
```


| 17. Further Reading

下面 5 份官方资料最值得继续读：

1. **Getting Started**: 最短上手路径，第一次安装从这里开始。
2. **Walkthrough**: 把 tools、env 和 tasks 连起来看，适合完整走一遍。
3. **`mise.lock` Lockfile**: 团队复现、checksum、URL 与 provenance 的核心资料。
4. **Task Configuration**: 任务系统的完整字段参考。
5. **Troubleshooting**: PATH、缓存、CI 和 Shell 问题的官方排查清单。

完整来源、文档漂移和边界说明见 `sources.md` 。日常命令速查见 `CHEATSHEET.md` 。

| 附录：mise Cheatsheet

这一章可以独立打印。第一次接入按顺序执行，日常使用按问题查命令。

1. 安装

```
# macOS
brew install mise

# Linux/macOS quick install
curl https://mise.run | sh

# Windows
scoop install mise

# Verify
mise --version
# 或 ~/.local/bin/mise --version
```

2. Shell 激活

```
# bash
echo 'eval "$(mise activate bash)"' >> ~/.bashrc

# zsh
echo 'eval "$(mise activate zsh)"' >> ~/.zshrc

# fish
echo 'mise activate fish | source' >> ~/.config/fish/config.fish

# Verify setup
mise doctor
```

交互 Shell 优先 `mise activate`。CI、IDE 和脚本优先 `mise exec`、`mise run` 或 `shims`。

3. 最常用命令

目标	命令
为项目安装并启用 Node.js	<code>mise use node@26</code>
设置全局默认 Node.js	<code>mise use --global node@26</code>
安装项目声明的所有工具	<code>mise install</code>
临时运行特定 Python	<code>mise exec python@3.14 -- python</code>
在项目环境中执行命令	<code>mise exec -- npm test</code>
运行任务	<code>mise run test</code>
查看当前工具	<code>mise ls --current</code>
查看工具详情	<code>mise tool ripgrep</code>
查找真实可执行文件	<code>mise which node</code>
查看可用远端版本	<code>mise ls-remote node</code>
查看可升级工具	<code>mise outdated</code>
范围内升级	<code>mise upgrade</code>
提升主版本范围	<code>mise upgrade --bump node</code>
查看加载的配置	<code>mise config</code>
诊断安装	<code>mise doctor</code>
清除版本缓存	<code>mise cache clear</code>

4. 新项目最小配置

```
# mise.toml
[tools]
node = "26"
python = "3.14"

[env]
NODE_ENV = "development"

[tasks.test]
run = "npm test"
```

```
mise trust
mise install
mise run test
```

5. use、install、exec、run

命令	安装工具	写 mise.toml	加载项目环境
mise use node@26	是	是	是
mise install	是	否	安装阶段
mise exec -- <cmd>	按需	否	是
mise run <task>	按需	否	是

新增工具用 `mise use`。clone 项目后用 `mise install`。脚本和 CI 用 `mise exec` 或 `mise run`。

6. 配置层级

```
~/.config/mise/config.toml      # 用户全局
~/work/mise.toml                # 工作目录共享
~/work/project/mise.toml        # 项目共享
~/work/project/mise.local.toml  # 项目本地, 不提交
```

查看真实加载顺序:

```
mise config
mise config ls
```

环境专用配置:

```
mise.toml
mise.test.toml
mise.local.toml
mise.test.local.toml
```

```
MISE_ENV=test mise run test
mise -E test run test
```

7. 工具版本

```
[tools]
node = "26"           # 允许 26.x
python = "3.14"        # 允许 3.14.x
ruby = "latest"        # 每次解析最新可用版本
go = "prefix:1.25"     # 允许 1.25.x
```

常用后端显式写法：

```
mise use github:BurntSushi/ripgrep
mise use aqua:aws/aws-cli
mise use npm:prettier
mise use pipx:black
mise use cargo:starship
```

默认使用 Registry shorthand；需要显式选择后端时，使用 `aqua:`、`github:` 或 `gitlab:`。对 `asdf:` 插件保持审慎。

8. 环境变量

```
[env]
NODE_ENV = "development"
_.path = "./node_modules/.bin"
_.file = ".env"
```

多个文件：

```
[env]
_.file = [
  ".env",
  ".env.local",
  { path = ".secrets.yaml", redact = true },
]
```

Required variables：

```
[env]
DATABASE_URL = { required = "Set DATABASE_URL in mise.local.toml" }
API_KEY = { required = "Get API_KEY from the team secret manager" }
```

Redaction：

```
redactions = ["SECRET_*", "*_TOKEN", "PASSWORD"]

[env]
API_TOKEN = { value = "token-value", redact = true }
```

CLI:

```
mise set NODE_ENV=development
mise set
mise env
mise env --redacted
mise unset NODE_ENV
```

9. Secret 边界

共享 `mise.toml` 不写真实 secret。

推荐来源：

1. `mise.local.toml`
2. 系统环境变量
3. 组织 secret manager
4. CI 平台 secret

`.gitignore` :

```
mise.local.toml
mise.local.lock
mise*.local.toml
mise*.local.lock
.env
.env.local
```

10. 任务

简单任务：

```
[tasks]
build = "npm run build"
test = "npm test"
lint = "npm run lint"
```

详细任务：

```
[tasks.test]
description = "Run tests"
run = [
  "npm test",
  "uv run pytest",
]
```

依赖并发：

```
[tasks.ci]
description = "Run CI checks"
depends = ["lint", "test", "build"]
```

输入输出缓存：

```
[tasks.build]
run = "npm run build"
sources = ["src/**/*.ts", "!src/**/*.test.ts", "tsconfig.json"]
outputs = ["dist/**/*.js"]
```

监视文件：

```
mise use --global watchexec@latest
mise watch build
```

危险任务确认：

```
[tasks.deploy]
confirm = { message = "Deploy production?", default = "no" }
run = "./scripts/deploy.sh"
```

注意：confirm 不会阻止 depends 在提示前执行。

11. mise.lock

启用：

```
mise settings lockfile=true
mise lock
mise install
git add mise.toml mise.lock
```

项目配置：

```
[settings]
lockfile = true
```

Strict mode:

```
MISE_LOCKED=1 mise install
```

多平台:

```
mise lock --platform linux-x64,macos-arm64
```

团队提交:

```
提交:mise.toml、mise.lock、mise.<env>.toml、mise.<env>.lock
忽略:mise.local.toml、mise.local.lock、mise.<env>.local.toml、
mise.<env>.local.lock
```

12. GitHub Actions

截至 2026-06-02, `jdx/mise-action` 最新 release 是 `v4.0.1` :

```
name: test
on:
  pull_request:
  push:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v6
      - uses: jdx/mise-action@v4
      - run: mise run ci
```

仓库存在 `mise.lock` 时, `mise-action` 会自动执行 `mise install --locked` 。

13. CI 和脚本

推荐:

```
mise install
mise exec -- npm test
mise run ci
```


Shims:

```
export PATH="$HOME/.local/share/mise/shims:$PATH"
```

不要在非交互脚本中依赖 prompt 刷新 PATH。

14. 安全设置

Trust:

```
mise trust
mise trust --show
mise untrust
```

Paranoid mode:

```
mise settings paranoid=1
MISE_PARANOID=1 mise install
```

最小发布时间:

```
[settings]
minimum_release_age = "7d"
```

Hooks 带实验性标记。默认优先显式任务:

```
mise run setup
```

15. asdf 迁移

```
# 保留现有 .tool-versions
mise install

# 需要时生成 mise 配置
mise config generate

# 启用 lockfile
mise settings lockfile=true
mise lock
```

mise 能读 `.tool-versions`，但不会直接复用 asdf 数据目录。

16. direnv 边界

direnv 和 mise 都会通过 Shell hook 管理环境。不要让两者同时管理同一套 PATH，尤其是 Python virtualenv 和运行时版本。

17. 排障

```
# 总体诊断
mise doctor
mise --version
mise config
mise ls --current

# PATH
which -a node
mise which node
mise ls node

# 缓存
mise cache clear
mise ls-remote node

# 调试日志
MISE_DEBUG=1 mise install
MISE_TRACE=1 mise install

# Prompt 性能
mise deactivate
MISE_TIMINGS=1 mise hook-env -s bash 2>&1 >/dev/null
```

18. 推荐项目模板

```
min_version = "2026.5.18"

[settings]
lockfile = true
minimum_release_age = "7d"

[tools]
node = "26"
python = "3.14"
uv = "latest"
ripgrep = "latest"

[env]
NODE_ENV = "development"
DATABASE_URL = { required = "Set DATABASE_URL in mise.local.toml" }
_.path = "./node_modules/.bin"

[tasks.setup]
run = [
    "npm install",
    "uv sync",
]

[tasks.lint]
run = [
    "npm run lint",
    "uv run ruff check .",
]

[tasks.test]
run = [
    "npm test",
    "uv run pytest",
]

[tasks.ci]
depends = ["lint", "test"]
```

附录：资料来源与边界

教程中的版本、命令与边界以 `mise` 官方仓库、官方文档和 GitHub Releases API 为准。交叉验证只使用对应项目的官方文档。

Method

本手册优先使用 `mise` 官方仓库、`mise` 官方文档站和 GitHub Releases API。为了核对迁移、PATH 与 CI 相关判断，补充使用 `asdf`、`direnv` 和 GitHub Actions 官方文档。正文中的建议分为三类：

- 官方事实：命令行为、配置字段、文件优先级、实验性状态和 release 版本。
- 工程建议：基于官方事实整理出的推荐顺序，例如交互 Shell 用 `mise activate`，脚本和 CI 优先用 `mise exec`、`mise run` 或 `shims`。
- 边界提醒：不能从工具文档推出的结论，例如组织内部 secret 管理、合规要求和发布权限。

官网页面快照保存在 `sources/raw/`，从官方仓库复制的 Markdown 原稿保存在 `sources/official-docs/`。正文中的命令以官方仓库提交 `310e325909893b6af5fb1aa6a42653eaa7f35131` 和 GitHub API 在 2026-06-02 返回的信息为准。

Current Versions

- `mise` 最新稳定版：v2026.5.18，发布于 2026-05-31T21:43:15Z
- `jdx/mise-action` 最新 release：v4.0.1，发布于 2026-03-22T16:06:57Z

Documentation Drift

`mise` 官网 CI 页面仍展示 `jdx/mise-action@v3`，而 `jdx/mise-action` 仓库 README 已展示 `jdx/mise-action@v4`。本文以 action 仓库为准。

`tips-and-tricks.md` 仍出现通过 `touch mise.lock` 启用锁文件的旧式示例。当前专门的 `lockfile` 文档提供 `mise lock`，本文采用 `mise lock`。

Primary Sources: mise

- GitHub repository: <https://github.com/jdx/mise>
- Latest release API: <https://api.github.com/repos/jdx/mise/releases/latest>
- Official documentation: <https://mise.jdx.dev/>
- Getting Started: <https://mise.jdx.dev/getting-started.html>
- Installing mise: <https://mise.jdx.dev/installing-mise.html>
- Walkthrough: <https://mise.jdx.dev/walkthrough.html>
- About: <https://mise.jdx.dev/about.html>
- Configuration: <https://mise.jdx.dev/configuration.html>
- Configuration Environments: <https://mise.jdx.dev/configuration/environments.html>
- Dev Tools: <https://mise.jdx.dev/dev-tools/>
- Backends: <https://mise.jdx.dev/dev-tools/backends/>
- Registry: <https://mise.jdx.dev/registry.html>

- `mise.lock` : <https://mise.jdx.dev/dev-tools/mise-lock.html>
- Shims: <https://mise.jdx.dev/dev-tools/shims.html>
- Environments: <https://mise.jdx.dev/environments/>
- Secrets: <https://mise.jdx.dev/environments/secrets/>
- Tasks: <https://mise.jdx.dev/tasks/>
- TOML Tasks: <https://mise.jdx.dev/tasks/toml-tasks.html>
- Running Tasks: <https://mise.jdx.dev/tasks/running-tasks.html>
- Task Configuration: <https://mise.jdx.dev/tasks/task-configuration.html>
- Continuous Integration: <https://mise.jdx.dev/continuous-integration.html>
- Hooks: <https://mise.jdx.dev/hooks.html>
- Trust CLI: <https://mise.jdx.dev/cli/trust.html>
- Paranoid mode: <https://mise.jdx.dev/paranoid.html>
- Tips & Tricks: <https://mise.jdx.dev/tips-and-tricks.html>
- Troubleshooting: <https://mise.jdx.dev/troubleshooting.html>
- Comparison to asdf: <https://mise.jdx.dev/dev-tools/comparison-to-asdf.html>
- Security policy: <https://github.com/jdx/mise/blob/main/SECURITY.md>

Primary Sources: CI and Cross-checks

- `jdx/mise-action` : <https://github.com/jdx/mise-action>
- `jdx/mise-action` latest release API: <https://api.github.com/repos/jdx/mise-action/releases/latest>
- asdf Introduction: <https://asdf-vm.com/guide/introduction.html>
- asdf Getting Started: <https://asdf-vm.com/guide/getting-started.html>
- direnv Shell Hook Setup: <https://direnv.net/docs/hook.html>
- GitHub Actions dependency caching: <https://docs.github.com/en/actions/concepts/workflows-and-actions/dependency-caching>
- GitHub Actions security: <https://docs.github.com/en/actions/how-tos/security-for-github-actions>

Boundaries

本文不是 `mise` 官方文档的替代品，也不承诺覆盖每个后端、插件和实验性配置。正式接入前，应复核目标操作系统、Shell、CI 平台、网络环境、组织 `secret` 管理规则和供应链策略。

`hooks`、`monorepo root`、`SOPS` 和 `direct age encryption` 在官方文档中带有实验性标记。本文会解释它们的用途，但不把它们放进默认配置。